

这些“函数除了返回值之外，还对外部世界产生的影响”就是副作用。

```
1 function UserList() {
2   const [users, setUsers] = useState<User[]>([]);
3   const [loading, setLoading] = useState(true);
4
5   useEffect(() => {
6     fetch('https://api.example.com/users')
7       .then(res => res.json())
8       .then(data => {
9         setUsers(data);
10        setLoading(false);
11      });
12   }, []); // ← 关键：空数组
13
14   if (loading) return <p>加载中...</p>;
15   return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
16 }
```

：空数组表示只在组件首次渲染后执行一次。

模式二：监听依赖变化

```
1 function SearchResults({ keyword }: { keyword: string }) {
2   const [results, setResults] = useState<string[]>([]);
3
4   useEffect(() => {
5     if (!keyword) return;
6
7     fetch(`/api/search?q=${keyword}`)
8       .then(res => res.json())
9       .then(data => setResults(data));
10   }, [keyword]); // ← 关键：声明依赖
11
12   return <ul>{results.map((item, i) => <li key={i}>{item}</li>)}</ul>;
13 }
```

🔗 ☆ @

挂载执行	<code>[]</code>	组件挂载时执行一次	初始化数据获取
监听变化	<code>[a, b]</code>	挂载 + 依赖变化时执行	响应特定值变化
每次执行	省略	每次渲染后都执行	几乎不用，危险

清理函数：防止内存泄漏

问题：组件卸载时，定时器/事件监听还在运行 → 内存泄漏

解决：返回清理函数

```
1 // 定时器清理
2 useEffect(() => {
3   const timer = setInterval(() => {
4     console.log('每秒执行');
5   }, 1000);
6
7   return () => clearInterval(timer);
8 }, []);
9
10 // 事件监听清理
11 useEffect(() => {
12   const handleResize = () => setWidth(window.innerWidth);
13   window.addEventListener('resize', handleResize);
14
15   return () => window.removeEventListener('resize', handleResize);
16 }, []);
```

依赖数组的常见陷阱

陷阱1：遗漏依赖

```
1 // ❌ 错误：使用了 count 但没写进依赖
2 useEffect(() => {
3   console.log(count);
4 }, []); // count 永远是初始值
5
6 // ✅ 正确
7 useEffect(() => {
8   console.log(count);
9 }, [count]);
```

不会重新触发

陷阱2：闭包陷阱

```
1 // ❌ 错误：捕获了初始值
2 ▼ useEffect(() => {
3 ▼   const timer = setInterval(() => {
4     setCount(count + 1); // 永远是 0 + 1
5   }, 1000);
6 }, []);
7
8 // ✅ 正确：使用函数式更新
9 ▼ useEffect(() => {
10 ▼   const timer = setInterval(() => {
11     setCount(prev => prev + 1); // 基于最新值
12   }, 1000);
13   return () => clearInterval(timer);
14 }, []);
```

函数式更新 `set(p=>p+1)`

```
// main.tsx
<BrowserRouter>
  <App />
</BrowserRouter>
// App.tsx
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="/users/:id" element={<UserProfile />} />
  <Route path="*" element={<NotFound />} />
</Routes>
```

- `<a>` 标签会刷新整个页面
- `<Link>` 只切换组件 (SPA 体验)

```
1 // useToggle
2 function useToggle(initialValue = false) {
3   const [value, setValue] = useState(initialValue);
4   const toggle = () => setValue(v => !v);
5   return [value, toggle];
6 }
7
8 // useLocalStorage
9 function useLocalStorage<T>(key: string, initialValue: T) {
10  const [value, setValue] = useState<T>(() => {
11    const item = localStorage.getItem(key);
12    return item ? JSON.parse(item) : initialValue;
13  });
14
15  const setStoredValue = (val: T) => {
16    setValue(val);
17    localStorage.setItem(key, JSON.stringify(val));
18  };
19
20  return [value, setStoredValue];
21 }
```

1. 项目背景

1.1 问题陈述

当前企业内部缺乏统一的任务管理工具，员工使用 Excel 或纸质便签记录待办事项，导致：

- 任务状态不同步，团队协作困难
- 历史记录无法追溯，复盘依赖记忆
- 多端切换体验割裂（电脑/平板/手机）

1.2 目标

开发一款轻量级 Web 待办应用，支持个人任务管理与简单团队协作，具备以下特性：

- 数据本地持久化，刷新不丢失
- 响应式设计，适配桌面与平板
- 支持深色模式，减少眼部疲劳

1.3 成功指标

指标	目标值	验证方式
页面首屏加载时间	< 1.5s	Lighthouse Performance
任务操作响应时间	< 100ms	Chrome DevTools
数据持久化成功率	100%	自动化测试

2.2 功能需求

US-001 任务创建

作为用户，我需要快速添加任务，以便记录新的工作内容。

验收标准：

- 输入框支持回车快捷键提交
- 空任务禁止提交（前端校验）
- 添加成功后输入框自动清空并聚焦

US-002 任务状态切换

作为用户，我需要标记任务完成/未完成，以便跟踪工作进度。

验收标准：

- 点击任务行任意位置（除删除按钮外）可切换状态
- 已完成任务显示视觉差异（删除线、不同颜色）
- 切换状态时有平滑过渡动画

US-003 任务筛选

作为用户，我需要按状态筛选任务，以便快速查看待完成工作。

验收标准：

- 支持三种筛选条件：全部 / 未完成 / 已完成
- 当前筛选条件高亮显示
- 筛选结果实时更新，无需刷新页面
- 显示各状态下的任务数量

US-004 任务删除

作为用户，我需要删除不需要的任务，以保持列表整洁。

验收标准：

- 鼠标悬停时显示删除按钮

任务筛选可以删除

批量删除

数据自动保存

多页面

主题切换功能

首页+功能页+关于页

1782888638053

技术域	选型	说明
框架	React 18	函数组件 + Hooks 模式
语言	TypeScript	严格模式，类型安全
构建	Vite	快速冷启动，HMR
路由	React Router v6	声明式配置
样式	CSS 变量 + 原生 CSS	支持动态主题
存储	localStorage	轻量级本地持久化